



# Vulnerability detection with graph enhancement and global dependency representation learning

Xuehai Jia<sup>1,2</sup> · Junwei Du<sup>2</sup> · Minying Fang<sup>2</sup> · Hao Liu<sup>2</sup> · Yuying Li<sup>2</sup> · Feng Jiang<sup>1,2</sup>

Received: 27 June 2024 / Accepted: 29 December 2024 / Published online: 5 January 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

## Abstract

Vulnerability detection is essential for protecting software systems from attacks. Graph neural networks (GNNs) have proven effective in capturing semantic features of code and are widely used for this purpose. Existing GNN-based methods typically merge multiple graphs and employ GNNs to learn syntactic and semantic relationships within code graph structures. However, these methods face a significant limitation: current code graph structures inadequately represent parameter dependencies and node type information, which are crucial for capturing vulnerability patterns. This inadequacy hampers the GNNs' ability to discern and characterize vulnerable code, thereby undermining effective vulnerability detection. Additionally, traditional GNN-based methods may lose long-distance dependency information during aggregation, which is vital for understanding the behavior and occurrence patterns of vulnerable code. Despite achieving state-of-the-art performance, existing GNN-based methods struggle to fully understand vulnerability behaviors and their potential impacts. To address these issues, this paper introduces VulDecgre, a novel vulnerability detection model comprising two components: (1) An enhanced code graph structure that fuses multiple graphs and relational edges to improve code representation. (2) A natural sequence-aware learning module that integrates code execution sequence information to enhance vulnerability detection. Extensive experiments on three public datasets and a self-collected large-scale real-world C/C++ dataset demonstrate that VulDecgre achieves superior performance in vulnerability detection.

**Keywords** Vulnerability detection · Graph representation learning · Program analysis · Software engineering

## 1 Introduction

In recent years, the significant increase in open-source software libraries has led to a rapid growth in software vulnerabilities. The ‘Vulnerability and Threat Trends Report 2023’<sup>1</sup> reported that the ‘National Vulnerability Database’ (NVD)<sup>2</sup> added 25,096 new vulnerabilities in 2022, a 25% increase from the 20,196 new vulnerabilities reported in 2021. These vulnerabilities mainly stem from security threats within program code. They allow attackers to exploit programs to launch attacks on operating systems and network services, causing economic and social losses. Therefore, studying source code vulnerability detection technology is necessary to protect software security and defend against network threats.

To mitigate these threats, researchers have developed static and dynamic analysis tools that primarily use rule-matching methods to identify known vulnerability patterns in code. Gao et al. (2018) proposed a general defect detection tool called Cobot, which detects system vulnerabilities through static analysis of program source code and rule discrimination. Cadar et al. (2008) proposed KLEE, which uses a dynamic analysis method based on symbolic execution to troubleshoot errors. Song et al. (2018) introduced Angr, which integrates static analysis and dynamic symbolic execution to automate the analysis of binary files. However, traditional static defect detection methods often rely on experts to manually construct defect patterns, while dynamic detection methods face issues such as path explosion, constraint-solving difficulties, and parallel processing complexity. Additionally, relying solely on rule matching makes it difficult to detect unknown data and new vulnerabilities. Research has shown that deep learning-based vulnerability detection methods have demonstrated significant advancements in identifying new types of vulnerabilities (Li et al. 2018b; Cheng et al. 2021; Xiao et al. 2024). These approaches leverage natural language processing technologies to extract features from source code in the form of language sequences, producing classification results. Unlike static and dynamic analysis tools that target specific types of vulnerabilities, deep learning-based methods can identify unknown vulnerabilities by learning patterns of vulnerabilities.

Graph Neural Networks (GNNs) have received significant attention in recent years and have demonstrated their ability to capture the semantic features of code (Ahmad et al. 2021; Feng et al. 2020). They achieve this by aggregating nodes and edges on a graph, encapsulating complex logical relationships and structural information of code. A typical approach involves using heterogeneous graph technology to merge multiple types of graphs with abstract syntax trees (AST) as the basic structure and employing GNNs to learn the relationships within graphs that contain rich syntactic and semantic associations of the code. AST can preserve the syntax structure of the code in tree form and serve as the basic structure of a heterogeneous graph to connect other relational edges. This approach has proven effective in capturing detailed information within the code structure (Dam et al. 2019;

<sup>1</sup> <https://www.skyboxsecurity.com/resources/report/vulnerability-threat-trends-report-2023>.

<sup>2</sup> <https://nvd.nist.gov>.

Ghaffarian and Shahriari 2021). Multi-heterogeneous graph fusion methods provide a general framework for code representation learning. Zhou et al. (2019) and Dam et al. (2019) exploited the structural benefits of various code graph structures like the AST, control flow graph (CFG), control dependency graph (CDG) and natural code sequence (NCS), adding diverse relational edges such as jump edges to represent code. However, there are two challenges with the above approach. First, these methods face challenges in capturing the complex, deep parameter dependencies and node type information essential for identifying features of vulnerable code. Vulnerabilities such as command injection in operating systems and integer overflow are closely related to the types of defect functions and variables, so parameter dependency in function calls and variable node types are crucial for neural networks to learn the characteristics of vulnerabilities (Zhang et al. 2023). Thus, our goal is to accurately represent these two relationships within the structure of our graph. The second challenge is that directly learning the structure of a code graph with GNNs may result in the loss of significant sequence-related global dependency information, which is crucial for a comprehensive understanding of vulnerability behaviors and their potential impacts (Liu et al. 2023). Therefore, we aim to explore how sequence information can be integrated to enhance the model's understanding of code execution flow.

To address these two issues, we propose a novel deep learning-based vulnerability detection model to accurately detect vulnerabilities in source code functions. First, we connect the control flow edges and control dependency edges to the corresponding nodes in the AST structure. We also integrate four relationship edges (i.e., ComputeFrom, LastLexicalUse, NextToken, and GuardedBy) that have been proven effective in previous studies for representing specific vulnerability patterns (Liu et al. 2023), along with two new types of edges (i.e., ParameterDependency and TypeInformation) specifically defined to detect vulnerabilities related to parameter dependencies and node type information. Finally, we incorporate the above information into the nodes corresponding to the AST and propose an extended novel AST code graph structure, which we name 'NAST'. Second, we propose a gated graph neural network module, NS-GGNN, which integrates code natural sequences to achieve high-performance detection of multiple vulnerabilities.

Our model consists of two major components: (1) *A novel code enhancement graph structure* based on AST, which integrates the advantages of AST, CFG, and CDG, and incorporates edge relationship information that significantly impacts vulnerability detection to achieve a comprehensive representation of the vulnerability code structure; (2) *A Natural sequence-aware learning module*, which integrates the natural code sequence (NCS) information into code graph representations. NCS is constructed by connecting all adjacent leaf tokens in the AST, and it sequentially captures the logic of the source code in the form of tokens. Additionally, we designed a convolution module that incorporates a maximum pooling graph aggregation method, which more effectively filters node features and hidden states relevant to vulnerabilities, achieving better vulnerability detection results. We call this model VulDecgre (Vulnerability Detection via code graph representation Learning).

To evaluate VulDecgre, we use three widely studied benchmark datasets in software vulnerability detection: Fan et al. (2020), Reveal (2021) and FFMpeg+Qemu (2019). We compare VulDecgre against six existing software vulnerability detection methods. Additionally, existing open-source vulnerability datasets have several issues, such as an unbalanced proportion of vulnerability samples, unclear types of vulnerabilities, and an insufficient total number of vulnerabilities, which affect the effectiveness of vulnerability detection. We also use a self-collected dataset comprising C/C++ language source code functions, containing 18 types of vulnerabilities and a total of 330,207 positive and negative samples. The results demonstrate that VulDecgre outperforms all baseline methods in terms of accuracy and F1 score metrics. In particular, VulDecgre achieves absolute improvements in the F1 score of 0.43%, 5.46%, and 3.97% over the state-of-the-art on the three benchmark datasets. We conducted extensive experiments on our dataset to demonstrate that VulDecgre performed best in every experiment.

In summary, the significant contributions of this paper are as follows:

1. We propose a new code graph structure that captures the deep dependencies in code vulnerabilities, especially those affected by parameter dependencies and variable types.
2. We propose a novel graph representation learning module, NS-GGNN, which effectively integrates the natural sequential information of code to enhance the identification of essential features in vulnerability detection.
3. We built a dataset of real-world C vulnerability functions, including samples collected from ‘Software Assurance Reference Dataset’ (SARD),<sup>3</sup> NVD, and open-source projects on GitHub.<sup>4</sup> Our evaluations of three public benchmark datasets and a self-built dataset demonstrate that VulDecgre has significant advantages over existing methods, confirming its effectiveness in detecting vulnerable functions.

## 2 Background and motivation

In this section, we first briefly introduce the background of graph neural networks. Next, we describe the multi-head attention mechanism employed in VulDecgre, followed by a detailed explanation of the motivation behind VulDecgre’s design.

### 2.1 Graph neural networks

The graph structure of code is the key to representing code’s syntax and semantic information. Graph Neural Networks (GNNs) have attracted wide attention over the past few years, since GNNs can capture the structural information of code in the

---

<sup>3</sup> <https://nvd.nist.gov>.

<sup>4</sup> <https://github.com>.

software vulnerability detection task (Wen et al. 2023; Wang et al. 2023; Wen et al. 2024). Generally, a directed graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  contains a node list  $\mathcal{V}$  and a edge list  $\mathcal{E}$ , where  $(u, v) \in \mathcal{E}$  denotes an edge from the node  $u$  to the node  $v$ . GNNs aim to learn a representation of a node or an entire graph by passing information between nodes. Formally, GNNs learn a function  $f$  that updates the representation of each node based on its features and the features of its neighbors, as follows:

$$h_v^{(k)} = f(h_v^{(k-1)}, \{h_u^{(k-1)} : u \in \mathcal{N}(v)\}), \quad (1)$$

where  $h_v^{(k)}$  is the feature representation of the node after the  $k$ -round iteration.  $\mathcal{N}(v)$  denotes the set of neighbor nodes of  $v$ , and  $x = h_v^{(0)}$  is the initial feature of the node. This process iterates until a certain number of iterations is reached or the representations of nodes converge. The function  $f$  mentioned above typically distinguishes different GNN variants. For example, the message passing of gated graph neural networks GGNN (2015) can be expressed as:

$$h_v^{(k)} = \text{GRU}\left(h_v^{(k-1)}, \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{c_{v,u}} W h_u^{(k-1)}\right), \quad (2)$$

where  $a_v^{(k)} = \sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{c_{v,u}} W h_u^{(k-1)}$  is the aggregated message computed by summing over the transformed states  $W h_u^{(k-1)}$  of the neighboring node  $u$ , and  $c_{v,u}$  is a normalization constant, and  $W$  is the learnable weight. The GRU function then takes the previous state  $h_v^{(k-1)}$  and the aggregated message  $a_v^{(k)}$  as inputs to compute the new state  $h_v^{(k)}$ .

However, many code structure graphs have deep levels, and GGNNs tend to focus primarily on aggregating information from nearby nodes, struggling to handle long-range information from distant nodes during the iterative message-passing process. This issue is caused by the message-passing computation process (Li et al. 2018a; Oono and Suzuki 2019; Alon and Yahav 2020). This limitation may lead to the loss of dependencies between distant nodes during information aggregation. For example, the leaf nodes of the AST can record the execution order of the code, and when the AST is aggregated using GNNs, the execution order dependencies between distant nodes may be lost. To mitigate the loss of remote information and enhance graph learning capabilities, we propose a natural sequence-aware learning module that supplements graph learning with remote execution sequence information aggregation.

## 2.2 Multi-Head attention

Identifying vulnerabilities requires a deep understanding of the broad context of the code. By handling long-distance dependencies, multi-head attention helps models understand complex interactions across multiple code nodes. It is crucial for identifying security vulnerabilities. Multi-head attention is an essential component of the transformer architecture (Vaswani et al. 2017), frequently used in state-of-the-art NLP models. It extends the concept of self-attention and is designed to offer a

more complex and expressive attention mechanism. In the self-attention mechanism, each head computes attention scores using its query, key, and value. This is achieved using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (3)$$

where  $Q, K, V$  represent the query, key, and value respectively,  $d_k$  is dimensional length. To improve the performance of self-attention exceptionally on tasks requiring an understanding of the context in a sequence, multi-head attention projects  $h$  times input vectors (queries  $Q$ , keys  $K$ , and values  $V$ ) with different, learned linear projections to  $d_q, d_k$ , and  $d_v$  dimensions, as follows:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (4)$$

where each head  $\text{head}_i$  is computed as:

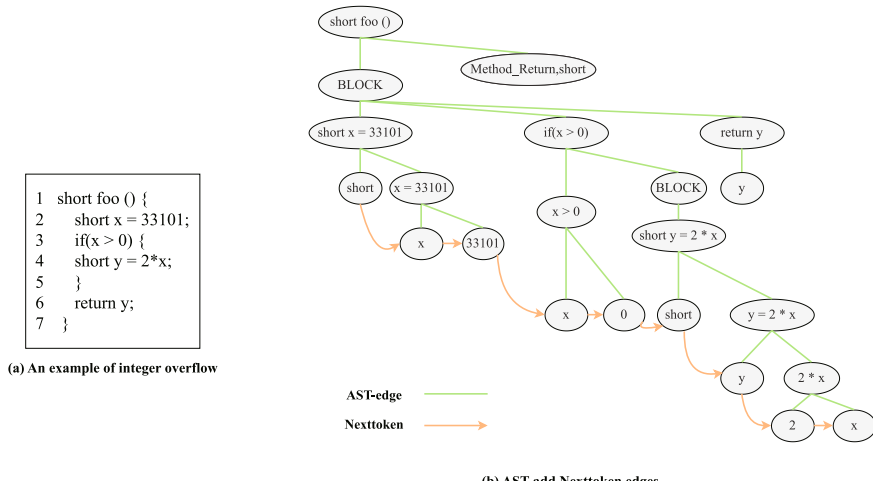
$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \quad (5)$$

where  $W_i^Q, W_i^K, W_i^V$ , and  $W^O$  are parameter matrices,  $h$  is the number of heads. This paper introduces a multi-head attention module that incorporates code execution sequence information to leverage long-distance dependencies in code.

## 2.3 Motivation

Unlike natural language, programming languages are more structured and hierarchical. They have various representation dimensions, such as AST, DFG, and CFG. Code representation based on multiple fusion graphs can capture structural and semantic information about programs. Zhou et al. (2019) proposed the Devign model, which advanced this field by encoding different levels of control flow, data dependencies, and natural code sequences into a joint graph for vulnerability detection tasks. Following this work, Wang et al. (2020) further confirmed that adding self-defined edges to the code graph structure can improve the performance of vulnerability detection tasks. In addition, Zhang et al. (2023), Tang et al. (2023), Peng et al. (2023), and other works have contributed to the integration of heterogeneous graphs and self-defined edge relationships. Inspired by these developments, we propose a new fusion heterogeneous graph, NAST, to adequately represent the structural and semantic information of code and to detect multiple types of vulnerabilities.

Moreover, due to the inherent characteristics of GNNs, it is difficult to obtain information from distant nodes during the message-passing process, resulting in a potential loss of code execution sequence relationships in the code graph. Specifically, we define the receptive field  $K$  as the distance from the farthest node that nodes on the graph can receive during neighborhood aggregation, and it significantly impacts the performance of GNNs. A large  $K$  may lead to overfitting or reduced model performance, whereas a small  $K$  may not capture enough context information for effective node representation. To illustrate, we consider a simple code example of integer overflow, as shown

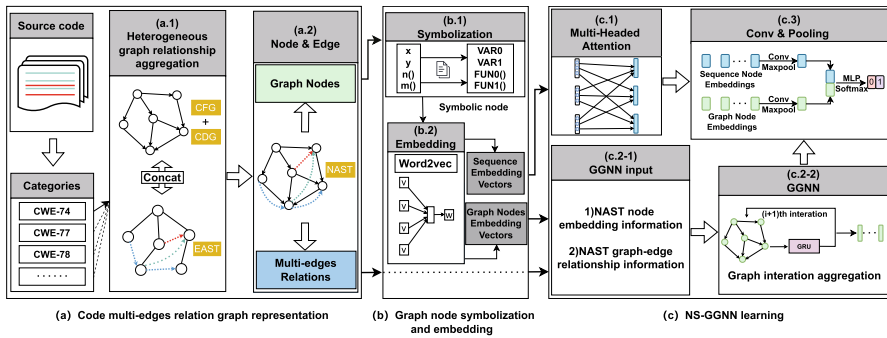


**Fig. 1** A real-world vulnerability function example, adding Nexttoken edges in AST to explain the limitation of GNNs in capturing remote dependency information

in Fig. 1. In Fig. 1a, the C integer overflow vulnerability function is represented as an AST, and the leaf nodes of the AST are connected from left to right, as shown in Fig. 1b. To capture the context between the leftmost node ‘short’ and the rightmost node ‘x’ in the AST when no leaf nodes are connected, the receptive field  $K$  must be set to at least 8. However, A large  $K$  may cause problems such as over-squashing and over-smoothing in GNNs. The  $K$  value usually selected often makes it challenging to realize the interaction of all sequence nodes. It is not conducive to the model’s understanding of the basic execution logic of program code. Therefore, in our study, we handle the leaf node sequence of the AST (i.e., Nexttoken sequence) separately to supplement the execution sequence interaction information between nodes for the restriction of  $K$ .

### 3 VulDecgre

In this section, we introduce our model VulDecgre, as shown in Fig. 2, which comprises three main processes: (1) Code multi-edges relation graph representation, which combines the multi-edges relationships in the code graph to enhance its structure for capturing deep semantic information. Step (a.1) represents the fusion process of multiple heterogeneous graphs and relational edges (i.e., the construction process of NAST), while step (a.2) involves obtaining the nodes and multilateral relations of the fused heterogeneous graphs for graph representation and learning. (2) Graph node symbolization and embedding, which involves assigning symbols to the code nodes and embedding all nodes and the leaf nodes of the AST. Step (b.1) represents the code symbolization process, and step (b.2) represents the embedding computation process for both types of nodes. (3) NS-GGNN learning, which integrates global execution information of the code into the learning process for graph representation, enhancing the model’s ability to capture global code interactions.



**Fig. 2** Architecture of VulDecgre model

Step (c.1) extracts multidimensional features of sequence node embeddings using multi-head attention. Step (c.2) consists of step (c.2-1) and step (c.2-2), learning node features of graphs through neighborhood aggregation and graph iteration aggregation. Step (c.3) uses graph pooling technology to aggregate the global features of the graph and integrate multidimensional embeddings to realize vulnerability detection.

### 3.1 Code graph construction

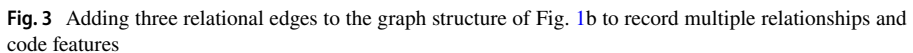
#### 3.1.1 Code graph and multi-edges fusion

The AST can adequately represent the abstract syntax structure of the code. Inspired by Wen et al. (2023), we extract the AST from a given code function to serve as the base structure of the graph. We then connect the nodes with control flow and control dependencies on the AST to fuse the AST, CFG, and CDG. Additionally, we aim to detect various types of vulnerabilities with distinct characteristics. We refer to methods previously studied by Wang et al. (2020), Liu et al. (2023), connecting the following four types of relationship edges, which are effective for vulnerability detection based on the fusion graph, to ensure high-performance detection of various types of vulnerabilities:

*ComputeFrom* connects variables on the left side of each assignment expression to all variables on the right side of the assignment statement. In Fig. 3, we connect the *y* variable node with the  $2 * x$  node using a *ComputeFrom* edge, indicating that the value of *y* is calculated from the value of *x*. These edges are critical for tracing variable values to identify vulnerabilities such as ‘CWE-476: Null Pointer Dereference’.

*LastLexicalUse* connects a variable in a conditional statement to its most recent usage. In Fig. 3, we connect the *x* variable from the conditional statement to its subsequent use, highlighting the relationship and aiding in detecting vulnerabilities such as ‘CWE-415: Double-Free’.

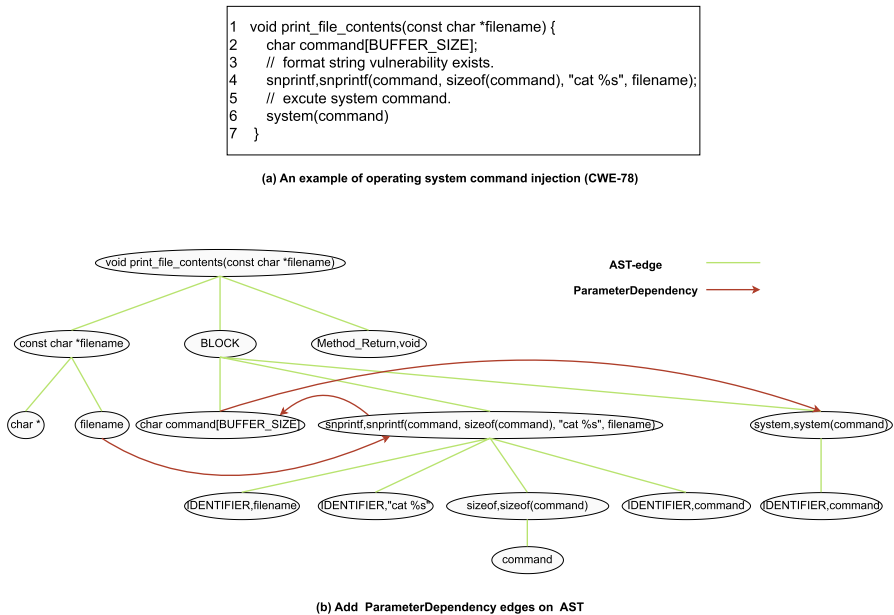




*Guardedby* and *GuardedbyNegation* form protective expressions by linking the node of a conditional expression with occurrences of the same variable within the statement governed by the condition, thus clarifying the code’s execution pathway under specific conditions. For instance, in Fig. 3, we employ a *Guardedby* edge to link the conditional expression  $x < MAX$  to the recurring  $x$  variable node in the subsequent subsequent statement. This association is crucial for monitoring the consistent use of the variable  $x$  in both the conditional expression and the subsequent statement, ensuring the model can learn these associations.

### 3.1.2 ParameterDependency and TypeInformation

 Springer



**Fig. 4** A real-world CWE-78 (Operating System Command Injection Vulnerability) example is shown in part **a**. Part **b** illustrates connecting the ParameterDependency edge to the AST to capture the parameter dependencies in the vulnerability function

function calls. For example, as shown in Fig. 4a, for the real-world ‘CWE-78: Operating System Command Injection’ vulnerability in the function `print_file_contents()`, we identify and connect three different parameter dependency edges: (1) from the `filename` parameter node in the function declaration to the `snprintf` function call node; (2) from the `snprintf` function call node to the local variable `command`’s definition node, which holds the formatted string returned by `snprintf`; (3) from the `command` variable node to the `system` function call node, since `command` is passed as an argument to `system`, instructing it to execute the `command` encoded in the string. These edges reveal how the `filename` parameter can become part of the `command` string and how the `command` string can be used as the parameter to a system call, potentially leading to command injection vulnerabilities. Thus, including *ParameterDependency* edges is crucial for capturing vulnerability patterns arising from function calls and specific parameter passing within functions.

For *TypeInformation*, we extract the data type information from three types of nodes by traversing the AST: function parameter, variable nodes within functions, and expression nodes. These relationships meticulously document the type attributes of parameter nodes, function return nodes, and expression nodes, thereby facilitating the detection of vulnerabilities associated with data type issues, such as ‘CWE-190: Integer Overflow’. This vulnerability often stems from parameters or expressions exceeding the maximum value their data type can accommodate, leading to an overflow. By extracting type information from these three node categories, we can recognize patterns indicative of such vulnerabilities. The extracted type information

equips the deep learning model to discern the data types associated with each node, consequently enhancing the precision in identifying potential vulnerabilities. Our experiments in Section 5.2 prove this point.

We then integrate the previously described four edges into the code graph structure and incorporate *ParameterDependency* and *TypeInformation* into the representation of the entire code graph, effectively highlighting the vulnerable parts of the code structure that are directly related to triggering vulnerabilities. This approach uses four relational edges to extract structural features commonly found in code and captures the features of vulnerabilities triggered by parameter dependencies and type information issues. These different code features interact effectively, enhancing the detection of both general and specific vulnerabilities. Finally, we built a novel extended AST, which we named 'NAST'. Figure 5 presents an expanded NAST that incorporates the relational edges from Fig. 2b, capturing a more comprehensive array of node and edge interactions within the code graph compared to a singular graph structure. Ultimately, the directed graph with multiple edge types can be represented as  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , where the graphs share the same node set  $\mathcal{V} = \mathcal{V}^{xst}$  and consist of six types of relational edges. Here,  $\mathcal{E}$  denotes the set of multiple edge types within the NAST.

### 3.2 Graph node symbolization and graph representation

After constructing the combined graph structure of the code, known as NAST and illustrated in Fig. 2b.1, we adopt the approach used by DeepWukong (2021) to standardize variable and function names in the NAST. This process normalizes each node into a uniform textual representation. Since 'Type' and 'Value' represent two distinct node properties within the code graph, they cannot be directly embedded into the nodes. Instead, we handle 'Type' and 'Value' separately. Regarding 'Type', we utilize a Word2vec (2017) model, pre-trained on a code corpus compiled during this research, to encode and obtain an embedded representation of the types of code nodes. For 'Value', we encode the code node values. We then concatenate these two embeddings to form the initial node representation, denoted as  $x_v$ . The AST is a tree structure composed of terminal and non-terminal nodes. Terminal nodes mainly contain information such as types, variables, and values in code statements. Connecting terminal nodes can effectively record the sequence of code execution. We process the terminal nodes of the AST in a left-to-right sequence. Specifically, for each node in the sequence, we use the same node embedding method mentioned above to get its initial embedding  $x_u$ .

To represent the edge relationships in the graph, we use an adjacency matrix to record the edge connections. Additionally, to enable bidirectional information flow between nodes, we represent the inverse relationships between nodes by applying the transposition of the matrix. This method considers information propagated on both forward and reverse paths, enhancing the model's ability to represent the data and improving communication between nodes in the code graph.

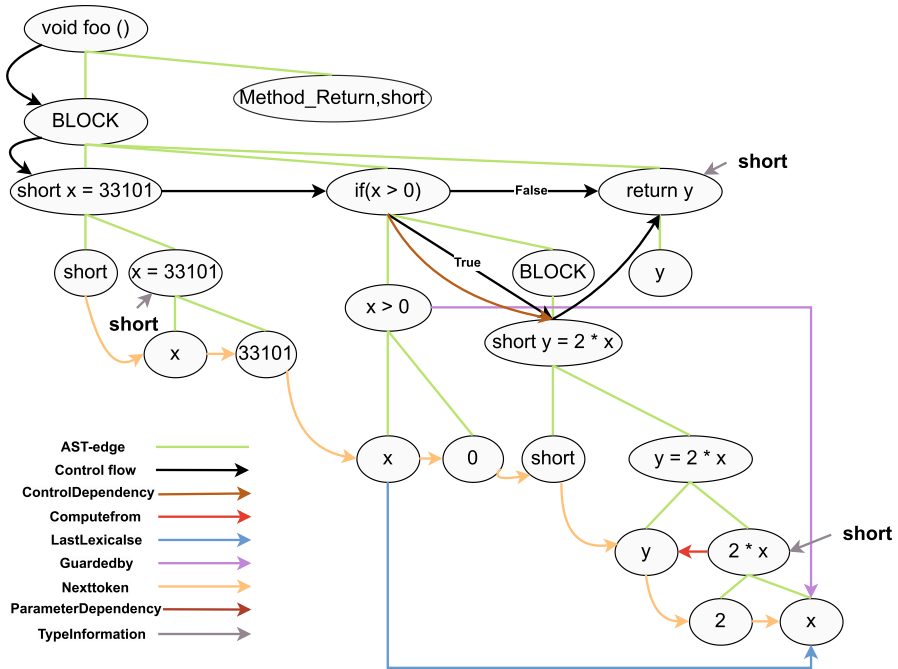


Fig. 5 The NAST of Fig. 1a which connects multiple relational edges to the AST

### 3.3 NS-GGNN module

To enhance graph representation learning, we propose a novel gated graph neural network (GGNN) module that incorporates natural code sequences (NCS) to learn global code embeddings. NCS edges connect adjacent leaf tokens in the AST. These tokens reserve the programming logic reflected by the sequence of source code. During node message passing, the GGNN utilizes Gated Recurrent Units (GRUs 2014) to effectively mitigate long-range dependency issues. Additionally, we integrate a multi-head attention mechanism for natural code sequence to enhance long-range dependency information that is easily lost during GGNN neighborhood aggregation.

#### 3.3.1 Gated graph recurrent layers

Given a directed program graph  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ , the GGNN learns node embeddings by processing both incoming and outgoing edge information. We create a composite adjacency matrix by combining the matrices of all incoming and outgoing edges in graph  $\mathcal{G}$ , denoted as  $A = [A^{(out)}, A^{(in)}] \in \mathbb{R}^{n \times 2n}$ . Assuming that  $D$  is the dimension of the node feature vector, we append a zero at the end of the initial feature vector to extend its dimension, thus accommodating high-dimensional semantic embedding operations. Subsequently, we aggregate the hidden state information from the

node at the previous time step with that of its neighboring nodes using the following equation:

$$a_v^{(t)} = A_v^{(t)} \left[ (h_1^{(t-1)})^T \dots (h_v^{(t-1)})^T \right]^T + b, \quad (6)$$

where  $a_v^{(t)} \in \mathbb{R}^{2D}$  represents the hidden information from node  $v$  and its neighbors, obtained through edge messaging and aggregation. The matrix  $A_v^{(t)}$  corresponds to the two columns in the adjacency matrix  $A$  representing the incoming and outgoing edges for node  $v$ . The GGNN processes all node information through bidirectional adjacency and the operation of node hidden states. It then calculates the candidate's hidden state and updates the hidden state as follows:

$$z_v^{(t)} = \sigma(W^z a_v^{(t)} + U^z h_v^{(t-1)}), \quad (7)$$

$$r_v^{(t)} = \sigma(W^r a_v^{(t)} + U^r h_v^{(t-1)}), \quad (8)$$

$$\hat{h}_v^{(t)} = \tanh(W a_v^{(t)} + U(r_v^{(t)} \odot h_v^{(t-1)})), \quad (9)$$

$$h_v^{(t)} = (1 - z_v^{(t)}) \odot h_v^{(t-1)} + z_v^{(t)} \odot \hat{h}_v^{(t)}, \quad (10)$$

where  $h_v^{(t-1)}$  denotes the hidden state of node  $v$  at the  $(t-1)$ -th step. The update gate  $z_v^{(t)}$  and the reset gate  $r_v^{(t)}$  regulate the amount of information to be retained or discarded by the node at the  $t$ -th step. The  $\sigma$  and  $\tanh$  functions are the sigmoid and hyperbolic tangent activation functions, respectively.  $W$  and  $U$  represent learnable weight matrices, while  $\odot$  signifies the Hadamard product. The reset gate  $r_v^{(t)}$  is applied to the previous hidden state  $h_v^{(t-1)}$  to determine the contribution of each element to the candidate hidden state  $\hat{h}_v^{(t)}$ . Although GGNN can effectively mitigate the partial loss of dependencies between nodes during the code graph learning process, it struggles to address the loss of long-range dependencies, such as the sequential connections between all AST leaf nodes.

### 3.3.2 Multi-Head attention layer

To enhance the model's ability to capture code sequence and other global interaction information, we design an attention module that incorporates the natural code sequence information, thereby augmenting the model's learning capabilities. Specifically, we employ a multi-head attention mechanism to learn the sequence features of code. This mechanism utilizes the initial embeddings of the sequence nodes and leverages the embedding matrices for the *Query*, *Key*, and *Value* to process the information:

$$h_{u_1} \dots h_{u_m} = \text{MultiHead}(\text{Query}, \text{Key}, \text{Value}), \quad (11)$$

where  $\text{Query}, \text{Key}, \text{Value} = (x_{u_1} \dots x_{u_m})$  are the embedding matrices corresponding to the sequence  $s = (s_{u_1} \dots s_{u_m})$  derived from the function slice  $c$ , with  $u_j$  representing

the  $j$ -th sequence node token of  $c$ . This process handles the sequence of AST leaf nodes that record the logic information of the source code, focusing on the relationships between long-range leaf nodes and capturing the key logic information of the code throughout the entire node sequence. This approach effectively alleviates the issue of long-range dependent information loss during GNN aggregation, enhances code representation, and increases the efficiency of vulnerability detection. We will verify these improvements in Section 5.2. Through processing, we obtain the graph node embeddings  $h_v$  and the sequence node embeddings  $h_u$ .

### 3.4 Output layer

In this section, we introduce the output layer of our model, which performs graph-level binary classification to determine whether a function is vulnerable. To enhance the predictive capability of our model, we improve the vulnerability output layer of the model by Zhou et al. (2019) by passing both the graph node embeddings  $h_v$  and sequence node embeddings  $h_u$  through a 1-D convolutional layer to filter code features. The filtered embeddings are denoted as:

$$Y_{v_i}^{(1)} = \sigma(h_{v_i}), \quad Y_{v_i}^{(2)} = \sigma(Y_{v_i}^{(1)}), \dots, Y_{v_i}^{(l)} = \sigma(Y_{v_i}^{(l-1)}), \quad (12)$$

$$S_{u_j}^{(1)} = \sigma(h_{u_j}), \quad S_{u_j}^{(2)} = \sigma(S_{u_j}^{(1)}), \dots, S_{u_j}^{(l)} = \sigma(S_{u_j}^{(l-1)}), \quad (13)$$

where  $\sigma(\cdot) = \text{ReLU}(\text{Conv}(\cdot))$ , and  $l$  is the number of convolutional layers. We then calculate the global embeddings for graph nodes and sequence nodes using a max-pooling layer:

$$h^g = \text{Maxpool}(Y_{v_i}^{(l)}, \forall v_i \in \mathcal{V}), \quad (14)$$

$$h^s = \text{Maxpool}(S_{u_j}^{(l)}, \forall u_j \in \mathcal{S}), \quad (15)$$

where  $\mathcal{V}$  is the graph node set, and  $\mathcal{S}$  is the sequence node set. Finally, we concatenate  $h^g$  and  $h^s$  to form  $h$  and pass this through a fully connected network. We apply the Softmax function to obtain the binary prediction result, indicating the presence or absence of a vulnerability:

$$\tilde{y}_i = \text{Softmax}(\text{MLP}(h)). \quad (16)$$

In conclusion, we analyze that the traditional GGNN has difficulty in capturing relationships between long-range AST leaf nodes that reflect the logical order of the source code during the neighborhood aggregation process for learning code graph node representations. These relationships are essential for GNNs to accurately understand code semantics. NS-GGNN effectively alleviates this limitation by incorporating interactions between enhanced logical relationship information and the overall code graph structure.

## 4 Experimental setup

### 4.1 Research questions

To evaluate VulDecgre, we answer the following research questions:

**RQ1:** How effective is VulDecgre in detecting vulnerabilities? We evaluated VulDecgre's vulnerability detection performance against the baseline model on three publicly available datasets and tested VulDecgre with other baseline models on 18 different types of vulnerabilities on a self-built dataset.

**RQ2:** How do the two types of self-defined relationships contribute to the performance improvement of VulDecgre? We evaluated the contribution of the two self-defined edges to VulDecgre's performance under the ten most typical vulnerability types.

**RQ3:** What are the advantages of the NS-GGNN module in vulnerability detection compared to the advanced graph learning modules? We compared the performance of NS-GGNN to three state-of-the-art graph-based neural network modules for vulnerability detection under the ten most typical vulnerability types.

**RQ4:** What is the influence of hyperparameters on the performance of VulDecgre? We evaluated the effects of the number of GGNN layers and the number of multi-head attention heads on VulDecgre's performance using our dataset.

### 4.2 Dataset

To assess the effectiveness of VulDecgre, we utilized three public datasets and one self-collected dataset. The public datasets are FFMpeg+Qemu (2019), Reveal (2021), and Fan et al. (2020). The statistical details of the datasets are presented in Table 1. The first column represents the category of the vulnerability dataset. The second, third, and fourth columns represent the total number of samples, the number of vulnerability samples, and the number of security samples, respectively. The fifth column shows the ratio of vulnerability samples. The FFMpeg+Qemu dataset comprises 22,961 samples with a vulnerability ratio of approximately 49%. The Reveal dataset consists of 158,399 samples, with 6.6% being vulnerabilities. Moreover, the Fan et al. dataset includes 188,636 samples, of which 5.7% are vulnerable.

As shown in Table 1, the vulnerability proportions in the three datasets are highly unbalanced, affecting the effectiveness of model detection. To address these issues, we collected datasets used in vulnerability detection research by Fan et al. (2023) and gathered code from open-source projects listed on SARD, NVD, and hosted on GitHub. We limited our collection to the top 25 most dangerous software vulnerabilities published by CWE between 2020 and 2023. Specifically, we reviewed the dataset published by Fan et al. and selected samples with fewer than 500 tokens to include in our dataset. We also collected vulnerability functions from SARD, NVD, and high-star open-source projects hosted on GitHub. Then, we selected the types of vulnerabilities with the largest number of samples as positive examples and used patched versions from SARD and NVD as negative (or non-vulnerable) samples. In the process of collecting vulnerability datasets, we face two main challenges: (1)

**Table 1** Statistics of three open source datasets

| Dataset           | Total samples | Vul samples | Non-vul samples | Vul ratio (%) |
|-------------------|---------------|-------------|-----------------|---------------|
| Reveal            | 158,399       | 10,493      | 147,906         | 6.62          |
| FFMPeg+Qemu       | 22,961        | 11,300      | 11,661          | 49.21         |
| Fan et al. (2020) | 188,636       | 10,900      | 177,736         | 5.77          |

the imbalance between positive and negative samples; and (2) the imbalance in the number of samples across different types of vulnerabilities. To ensure a balanced sample ratio in the dataset, we maintained a positive to negative sample ratio of 7:3. In addition, to address the problem of significant disparities in the total number of samples for different types within the collected dataset, we excluded types with fewer than 7000 samples. We believe that an insufficient number of samples can lead to an imbalance in the training process, resulting in biased detection outcomes. We also classified each vulnerability according to its corresponding CWE number, constructing a dataset of C source code functions. Our dataset is diverse, including several of the most common and significantly harmful types of vulnerabilities, such as overflow vulnerabilities, injection vulnerabilities, and resource management vulnerabilities. As depicted in Table 2, our dataset includes 18 categories of vulnerabilities, with approximately 30% of the samples being vulnerable. Compared to other datasets, our dataset features a higher percentage of vulnerable function samples, more sample vulnerabilities, and a greater diversity of vulnerability types.

### 4.3 Baseline methods

In our evaluation, We compared VulDecgre with three sequence-based methods from Li et al. (2018b, 2021b), Russell et al. (2018), eight state-of-the-art graph-based methods from Zhou et al. (2019), Chakraborty et al. (2021), Li et al. (2021a), Wu et al. (2022), Yuan et al. (2023), Cheng et al. (2022), Cao et al. (2024) and Wang et al. (2024). Since the detection granularity of VulDecgre is at the function level, even though some baseline methods can detect vulnerabilities across multiple functions, we only used them to detect vulnerabilities at the level of individual functions.

**IVDetect (2021):** IVDetect integrates five features of code statements and designs a feature attention GCN module to achieve statement-level vulnerability detection.

**VulDeePecker (2018):** VulDeePecker extracts code gadgets related to risk functions in the program and uses BiLSTM to learn them, achieving vulnerability detection across multiple functions. Here, we limit VulDeePecker to vulnerability detection at the level of individual functions.

**SySeVR (2021):** SySeVR designs an interprocedural slicing method that depends on the vulnerability statement and applies a bidirectional GRU network model to detect vulnerabilities. We also set its granularity to the level of individual functions.



**Table 2** Statistics of our self-built vulnerability function dataset

| CWE number    | Total samples | Vul samples | Non-vul samples | Vul ratio (%) |
|---------------|---------------|-------------|-----------------|---------------|
| CWE-074       | 39,309        | 11,491      | 27,818          | 29.23         |
| CWE-077       | 17,719        | 6110        | 11,609          | 34.48         |
| CWE-078       | 17,719        | 6110        | 11,609          | 34.48         |
| CWE-118       | 15,162        | 4402        | 10,760          | 29.03         |
| CWE-119       | 11,302        | 1862        | 9440            | 16.47         |
| CWE-120       | 30,992        | 10,306      | 20,686          | 33.25         |
| CWE-121       | 18,876        | 6329        | 12,547          | 33.53         |
| CWE-122       | 12,132        | 3985        | 8147            | 32.85         |
| CWE-125       | 11,696        | 3665        | 8031            | 31.34         |
| CWE-133       | 19,503        | 4839        | 14,664          | 24.81         |
| CWE-134       | 19,008        | 4719        | 14,289          | 24.83         |
| CWE-190       | 10,330        | 2528        | 7802            | 24.47         |
| CWE-191       | 13,130        | 3193        | 9937            | 24.32         |
| CWE-399       | 23,487        | 6675        | 16,812          | 28.42         |
| CWE-400       | 7151          | 1933        | 5218            | 27.03         |
| CWE-404       | 12,937        | 3875        | 9062            | 29.95         |
| CWE-704       | 12,427        | 4266        | 8161            | 34.33         |
| CWE-787       | 37,327        | 12,342      | 24,985          | 33.06         |
| Total Samples | 330,207       | 98,630      | 231,577         | 29.87         |

**Reveal (2021):** Reveal divides the training into two stages: the GGNN training module and the fully connected network module, to achieve better vulnerability detection.

**Russell et al. (2018):** Russell et al. convert the function code into a sequence, embed it into a matrix, and use convolutional neural networks and random forest classifiers for function-level vulnerability detection.

**Devign (2019):** Devign combines four code graph structures and uses gated graph neural networks to accurately detect vulnerability functions.

**VulCNN (2022):** VulCNN generates three vectors based on three types of centrality and uses a CNN to perform the classification task.

**VulBG (2023):** VulBG builds a behavior graph to represent the relationships between different functions and combines deep learning approaches to implement vulnerability detection.

**ContraFlow (2022):** ContraFlow is a path-sensitive code embedding approach that utilizes self-supervised contrastive learning to detect software vulnerabilities by preserving feasible value-flow paths and reducing the requirement for labeled data in training.

**Concoction** (2024): Concoction combines static code analysis with dynamic symbolic execution tracking and utilizes unsupervised active learning to reduce the overhead of symbolic execution, enabling efficient and accurate software vulnerability detection.

**Coca** (2024): Coca is a contrastive learning framework designed to enhance the robustness of GNN-based vulnerability detection models and provide concise and effective explanations for the detected vulnerabilities.

#### 4.4 Implementation details

To implement the VulDecgre model, we used the TensorFlow GNN framework and generated the composite graph structure of code with the Joern tool.<sup>5</sup> We used Word2Vec to initialize the node representations, each with an embedding vector dimension of 100. The model includes GNN layers with a hidden layer dimension of 128. Additionally, we employed the Neural Network Intelligence (NNI) framework to determine the optimal hyperparameters of the model through extensive fine-tuning experiments, thereby achieving optimal results.

During the model training process, we represented the three public datasets and one dataset with 18 types of vulnerabilities as the heterogeneous graph structure NAST, as shown in Section 3.2. We conducted offline training on the NS-GGNN module using batch samples. The dataset was randomly divided into training, validation, and test sets at a ratio of 8:1:1. We employed NNI to adjust the model's hyperparameters to achieve optimal performance. The training objective was to minimize the distance between predicted and real samples, and we utilized the cross-entropy loss function. We employed the Adam algorithm with a learning rate of 0.001. We trained our model for a maximum of 100 epochs on a server with an NVIDIA GeForce RTX 3090. Training was terminated if the loss was less than 0.001 or if 30 consecutive iterations failed to achieve better results.

#### 4.5 Performance metrics

We used the following four widely used performance metrics in our evaluation:

**Accuracy:** Accuracy is calculated by dividing the number of correctly classified samples by the total number of samples. It measures the overall correctness of a model's predictions. It is the proportion of correctly classified samples among the total number of samples:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP represents the number of true positives, TN is the number of true negatives, FP is the number of false positives, and FN is the number of false negatives.

<sup>5</sup> <https://joern.io/>.

However, Accuracy does not distinguish between types of errors, such as false positives and false negatives. This makes Accuracy less reliable for imbalanced datasets.

**Precision:** Precision is calculated by dividing the number of true positive predictions by the total number of positive predictions made by the model. It measures the accuracy of the model's positive predictions, quantifying the proportion of predicted positive samples that are truly positive:

$$Precision = \frac{TP}{TP + FP}$$

**Recall:** Recall is calculated by dividing the number of true positive predictions by the total number of actual positive instances. It measures the proportion of actual positive samples the model correctly identifies and evaluates the model's capacity to capture all relevant instances:

$$Recall = \frac{TP}{TP + FN}$$

**F1 score:** The F1 score is the Harmonic mean of Precision and Recall, providing a single metric that balances both Precision and Recall. It is beneficial in scenarios with an imbalanced class distribution:

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The Harmonic mean in the F1 Score formula ensures that the score only achieves a high value when both Precision and Recall are high. If either Precision or Recall is low, the F1 Score will be low as well.

## 5 Experimental results

### 5.1 RQ1: Effectiveness of VulDecgre

To address the RQ1, we conducted experiments with VulDecgre on three public datasets: FFMpeg+Qemu, Reveal, and Fan et al. We compared its performance with eight baseline methods detailed in Section 4.3. Furthermore, we assessed the detection capabilities of VulDecgre on a specially collected dataset comprising 18 types of vulnerabilities to determine the method's ability to detect a diverse set of vulnerabilities. The observations from these experiments were as follows.

#### 5.1.1 Comparison with baseline models

As shown in Table 3, we observe that graph-based methods outperform token-based methods in capturing the features of program functions. The three token-based methods (i.e., VulDeePecker, Russell et al., and SySeVR) perform worse than the graph-based methods across nearly all metrics. SySeVR uses a Bi-LSTM (Bidirectional

Long Short-Term Memory) network to address the long-distance dependency issue in code sequence tokens, achieving a maximum precision of only 40.07% on the Reveal dataset. While it is more effective than other token-based approaches, it is far less effective than graph-based methods like Devign and Reveal. We believe that the limitation of token-based methods lies in their inherent inability to effectively represent the code's syntactic structure and semantic information. Moreover, the two slice-based methods, VulDeePecker and SySeVR, are less effective than other methods across the three datasets. In addition to the limitations of the models themselves, the slice-based approaches retain key code information but fail to explore the graph structure information in the slicing code in depth, resulting in poor results. Among graph-based methods, we found that the Reveal method utilizing GGNN performed better on these three datasets than VulCNN and IVDetect, which use the traditional CNN method and the GCN method, respectively. Specifically, Reveal improves the results on the FFMpeg+Qemu dataset by 3.81%, 3.13%, 13.15%, and 7.35% compared to IVDetect across four metrics. IVDetect also performs better in a two-stage training process than Devign, which uses the original GGNN in a single-stage training process. Compared to other methods, the three models trained with contrastive learning, ContraFlow, Coca, and Concoction, achieve superior detection results, where Concoction performs the best. When evaluating Concoction, we found that approximately one-fifth of the code in the three open-source datasets was non-executable. This data was excluded to assess Concoction's detection performance.

Additionally, VulDecgre achieves the highest number of best metrics across three open-source datasets. Specifically, compared to VulDeePecker and SySeVR, VulDecgre achieves a 10.95–33.55% improvement in F1 scores across all three datasets. Compared to the traditional graph-based baseline method VulBG, VulDecgre achieves average improvements of 2.98%, 5.46%, and 3.97% in F1 score across the three datasets. Although VulBG uses a behavior graph to represent code behavior and achieves high vulnerability detection performance, the graph structure of VulDecgre captures more information related to vulnerability characteristics. Therefore, VulDecgre demonstrates superior vulnerability detection performance. Furthermore, compared to Concoction, the best-performing method based on contrastive learning, VulDecgre achieves F1 score improvements of 1.9%, 1.67%, and 0.08% across three different datasets. However, its Accuracy was lower than that of Concoction. We attribute this limitation to Concoction's hybrid approach, which combines dynamic and static representations to capture a more comprehensive range of the code's semantic information.

These results indicate that VulDecgre outperforms existing methods in vulnerability detection.

### 5.1.2 Detection performance of different types of vulnerabilities

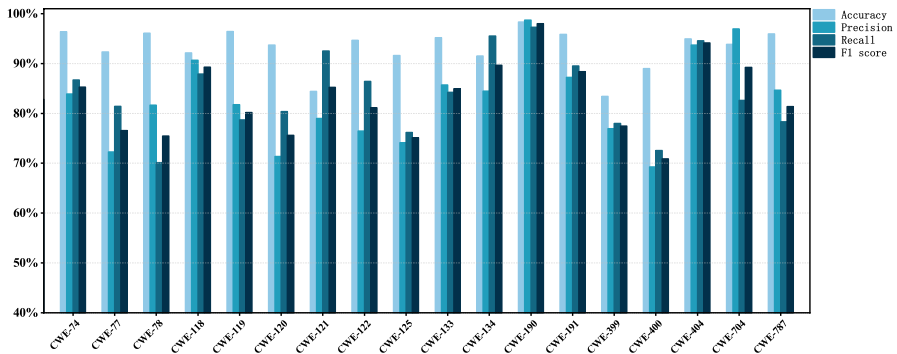
Figure 6 presents VulDecgre's detection performance across 18 different types of vulnerability samples. VulDecgre demonstrates high accuracy, precision, recall rates, and F1 scores for vulnerability detection. Specifically, using our method,

**Table 3** Comparison of different vulnerability detection methods on three open source datasets

| Baseline     | FFMPeg+Qemu  |             |              |              | Reveal       |              |              |              | Fan et al.   |              |              |              |
|--------------|--------------|-------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|              | Accuracy     | Precision   | Recall       | F1           | Accuracy     | Precision    | Recall       | F1           | Accuracy     | Precision    | Recall       | F1           |
| VulDeePecker | 49.61        | 46.05       | 32.55        | 38.14        | 76.37        | 21.13        | 13.1         | 16.17        | 81.19        | <b>38.44</b> | 12.75        | 19.15        |
| Russell      | 57.6         | 54.76       | 40.72        | 46.71        | 68.51        | 16.21        | 52.68        | 24.79        | 86.85        | 14.86        | 26.97        | 19.17        |
| SySeVR       | 47.85        | 46.06       | 58.81        | 51.66        | 74.33        | 40.07        | 24.94        | 30.74        | 90.1         | 30.91        | 14.08        | 19.34        |
| Devign       | 56.89        | 52.5        | 64.67        | 57.95        | 87.49        | 31.55        | 36.65        | 33.91        | 92.78        | 30.61        | 15.96        | 20.98        |
| Reveal       | 61.07        | <b>55.5</b> | 70.7         | 62.19        | 81.77        | 31.55        | <b>61.14</b> | 41.62        | 87.14        | 17.22        | 34.04        | 22.87        |
| VulCNN       | 60.74        | 50.97       | 60.27        | 55.2         | 83.49        | 24.1         | 41.83        | 30.58        | 85.35        | 26.89        | 25.7         | 26.28        |
| VulBG        | 61.21        | 54.94       | 65.2         | 59.63        | 88.98        | 39.3         | 50.66        | 44.26        | 86.63        | 31.29        | 25.11        | 27.86        |
| ContraFlow   | 58.74        | 53.19       | 61.42        | 57.01        | 84.53        | 35.69        | 55.41        | 43.42        | 92.44        | 30.75        | 32.17        | 31.44        |
| Coca         | 60.48        | 55.15       | 71.64        | 62.32        | 87.42        | 35.96        | 40.61        | 38.14        | 91.28        | 32.61        | 31.24        | <b>31.91</b> |
| Concocion    | <b>62.01</b> | 52.44       | 72.08        | 60.71        | <b>91.44</b> | 42.17        | 55.83        | 48.05        | <b>93.74</b> | 30.25        | 33.4         | 31.75        |
| IVDetect     | 57.26        | 52.37       | 57.55        | 54.84        |              |              |              |              |              |              |              |              |
| VulDecgre    | 61.39        | 53.44       | <b>75.58</b> | <b>62.61</b> | 90.39        | <b>45.62</b> | 54.64        | <b>49.72</b> | 93.58        | 27.66        | <b>37.49</b> | 31.83        |

Bold values indicate the maximum value of the current metric in all methods

This table compares the performance of different vulnerability detection methods on three open source datasets FFMPeg+Qemu, Reveal, and Fan et al.

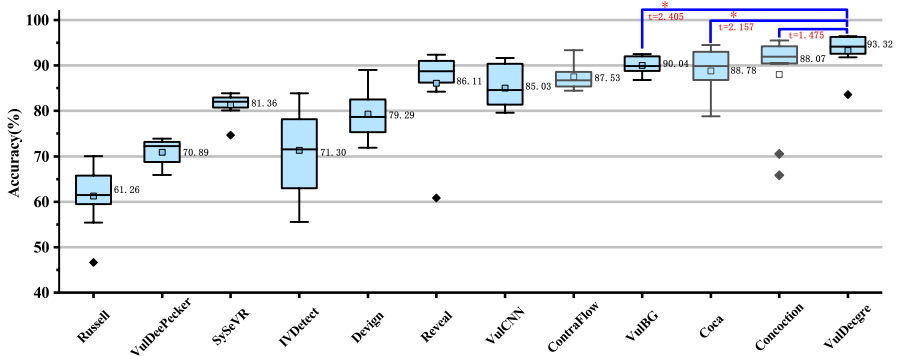


**Fig. 6** Comparison of VulDecgre model detection effect on 18 different types of vulnerabilities

the average accuracy, precision, recall, and F1 scores for these vulnerabilities are 93.32%, 82.15%, 83.49%, and 83.04%, respectively. Our method efficiently detects vulnerabilities characterized by clear parameter dependencies and type-related information, enabling accurate detection for most vulnerabilities. For example, VulDecgre performs best on the CWE-190 (Integer Overflow) vulnerability, achieving an F1 score of 97.98%. However, VulDecgre is not effective enough for specific categories, such as CWE-400 (Resource Consumption) vulnerabilities. We believe the reason is that the pattern of such vulnerabilities is unclear and depends on dynamic program behavior. It is difficult to capture the features of these vulnerabilities only based on static graph representation.

To provide a more accurate quantification of VulDecgre's detection performance, a box plot analysis was conducted for the accuracy of all methods across 18 types of vulnerabilities, as shown in Fig. 7. In general, the average accuracy of graph-based methods is higher than that of token-based methods. The mean accuracy of VulDecgre is the highest at 93.32%, VulBG has the second highest average accuracy at 90.04%, and IVDetect achieves the lowest accuracy among graph-based methods at 71.3%. SySeVR performs best among token-based methods, achieving an accuracy of 81.36%, which is even better than IVDetect and Devign. Moreover, the length of the box plots for VulDecgre, Concoction, VulBG, ContraFlow, and SySeVR is significantly smaller than those of other methods, indicating that the accuracy of these methods in detecting different types of vulnerabilities is more consistent and stable. IVDetect, Devign, and VulCNN have the longest box plots with a wide fluctuation range, indicating that the detection performance of these methods is not stable across different types of vulnerabilities.

As shown in Fig. 7, the box plot data reveal that VulDecgre represents a substantial improvement over the existing advanced methods, VulBG and Coca. For instance, the accuracy of VulDecgre is nearly 3.28% higher than that of VulBG and nearly 4.54% higher than that of Coca. A T-test was performed at a confidence level of 0.05 to assess the significance of VulDecgre compared to the three best-performing methods: VulBG, Coca, and Concoction. The T-values for VulDecgre and the other three methods are labeled on the graph. The T-values of



**Fig. 7** Box plots showing the accuracy of different vulnerability detection methods. The symbol ‘\*’ indicates the significance level of the detection effectiveness between the two methods across 18 types of vulnerabilities. \* indicates  $0.05 > T \text{ value} \leq 0.01$ , showing that there is a significant difference between the two sides of the comparison

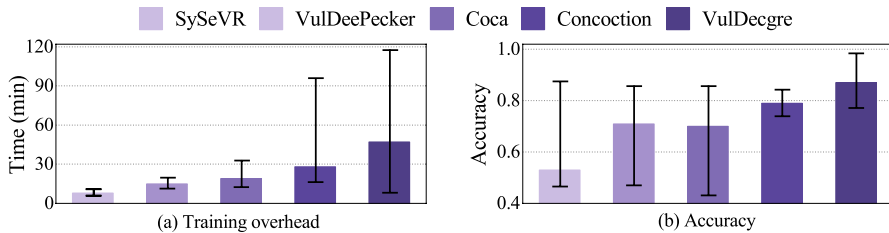
VulDecgre compared with VulBG and Coca are greater than 2.086, indicating a significant difference with their T-values at 0.05 level. Although the T-value of VulDecgre compared with Concoction is less than 2.086, which does not directly indicate a significant difference between them, VulDecgre demonstrates superior performance and stability. Its average Accuracy is higher than that of Concoction, and it achieves more consistent performance across multiple vulnerability types where Concoction’s detection performance is comparatively low.

In conclusion, the VulDecgre method outperforms other baseline models in detecting vulnerabilities. Its performance is significantly higher and more stable across 18 types of vulnerabilities compared to other models.

### 5.1.3 Model training overhead

Figure 8 shows the training overhead and the achieved accuracy for different vulnerability detection models. In the chart, In Fig. 8, the max–min boundary bars represent the highest and lowest performance values of the model over multiple tests, and the height of the bar chart shows the average performance of the tests. We conducted experiments on all 18 vulnerability datasets we collected. The training process was terminated when the loss did not improve for 30 consecutive training cycles or when the loss value fell below 0.01.

All the models can converge within 120 min on our training datasets. The token-based method SySeVR achieves minimal time expenditure, but it records the lowest detection accuracy and the highest level of fluctuation. Compared to SySeVR, the token-based method VulDeePecker achieves shorter training time and higher accuracy. The remaining methods require longer training time than SySeVR but deliver better detection accuracy. Compared to other methods, VulDecgre requires more training time because it processes more structural relationships during the code representation process, significantly increasing the training



**Fig. 8** The training cost and accuracy of different vulnerability detection models. The min-max bars displayed in the graph depict the range of model performance, showcasing the maximum and minimum values observed across multiple tests

overhead. However, the significant enhancement in detection accuracy justifies the additional time investment, demonstrating that this limited expenditure of time is indeed valuable.

## 5.2 RQ2: Effectiveness of two types of relationships

To answer RQ2, we added two types of relationships to the code graph structure in our experiments: *TypeInformation* and *ParameterDependency*. We selected ten of the most typical software security vulnerabilities to verify the contribution of these relationships to VulDecgre’s performance, as shown in Table 4. Adding only the *TypeInformation* relationship results in average increases of 2.39%, 0.66%, 3.79%, and 2% across four metrics. Adding the *ParameterDependency* relationship alone yields average increases of 2.6%, 3.28%, 3.38%, and 3.33%. When both relationships are incorporated, the performance improves by 4.43%, 3.88%, 6.51%, and 4.92%, demonstrating enhanced vulnerability detection.

When *TypeInformation* is added to the graph structure, the detection effectiveness for most types of vulnerabilities has been improved. Specifically, vulnerabilities related to data type representation range limits and memory structure boundary issues, such as Buffer Overflow (CWE-119), Classic Buffer Overflow (CWE-120), Integer Overflow (CWE-190), Out-of-bounds Read (CWE-125), and Out-of-bounds Write (CWE-787) vulnerabilities, significantly enhance detection performance. These vulnerabilities consistently appear in CWE 2023’s ‘Top 25 Most Dangerous Software Weaknesses’<sup>6</sup> and are among the most common. The detection ability for CWE-119 and CWE-787 vulnerabilities improves significantly, with F1 scores increasing by 8.9% and 9.5%, respectively. These improvements stem from the fact that indicated type can effectively imply the size of the variable and expression space used to identify undefined behavior (such as integer overflow) during the assignment process, enhancing the effectiveness of detection. Additionally, for vulnerability types not directly influenced by variables or buffer sizes, such as operating system command injection (CWE-78), adding *TypeInformation* led to a 2.92% improvement in F1 score performance. However, the improvement is minimal for

<sup>6</sup> <https://cwe.mitre.org/top25>.



**Table 4** Comparison of the effect of adding self-defined edges for ten types of vulnerabilities

| CWE-ID  | Original Graph |              |        |       | Original Graph+TypeInfo |              |              |              |
|---------|----------------|--------------|--------|-------|-------------------------|--------------|--------------|--------------|
|         | Accuracy       | Precision    | Recall | F1    | Accuracy                | Precision    | Recall       | F1           |
| CWE-74  | 95.14          | 85.6         | 83.37  | 84.47 | 94.77                   | 81.01        | 84.11        | 82.53        |
| CWE-77  | 89.61          | 71.48        | 79.52  | 75.29 | 89.16                   | 69.49        | 77.24        | 73.16        |
| CWE-78  | 93.49          | 78.82        | 72.72  | 75.65 | 93.8                    | 76.33        | 80.96        | 78.57        |
| CWE-119 | 90.6           | 75.32        | 68.19  | 71.58 | 93.72                   | 79.97        | <b>81</b>    | <b>80.48</b> |
| CWE-120 | 88.52          | 58.03        | 82.22  | 68.04 | 92.85                   | 60.26        | 83.72        | 70.08        |
| CWE-125 | 83.32          | 60.22        | 67.13  | 63.48 | 90.96                   | <b>68.81</b> | 66.01        | 67.38        |
| CWE-190 | 92.25          | 86.59        | 90.74  | 88.62 | 97.03                   | 90.44        | 90.65        | 90.54        |
| CWE-399 | <b>90.16</b>   | <b>81.69</b> | 79.45  | 80.55 | 87.28                   | 77.79        | 81.65        | 79.67        |
| CWE-400 | 88.33          | <b>73.37</b> | 62.99  | 67.78 | 89.33                   | 67.12        | 66.37        | 66.74        |
| CWE-787 | 87.49          | 75.11        | 71.7   | 73.37 | 93.86                   | 81.6         | <b>84.18</b> | 82.87        |
| Average | 89.89          | 74.62        | 75.80  | 75.2  | 92.28                   | 75.28        | 79.59        | 77.2         |

| CWE-ID  | Original Graph+ParameterDependency |              |              |              | Original Graph+2edges |              |              |              |
|---------|------------------------------------|--------------|--------------|--------------|-----------------------|--------------|--------------|--------------|
|         | Accuracy                           | Precision    | Recall       | F1           | Accuracy              | Precision    | Recall       | F1           |
| CWE-74  | 96.79                              | <b>87.24</b> | 83.47        | 85.31        | <b>97.37</b>          | 83.89        | <b>86.7</b>  | <b>85.27</b> |
| CWE-77  | <b>94.26</b>                       | <b>73.17</b> | <b>92.15</b> | <b>81.57</b> | 93.35                 | 72.25        | 81.43        | 76.57        |
| CWE-78  | 94.91                              | 80.71        | <b>74.57</b> | <b>77.51</b> | <b>97.09</b>          | <b>81.63</b> | 72.62        | 76.86        |
| CWE-119 | 94                                 | 81.35        | 73.82        | 77.4         | <b>97.44</b>          | <b>81.73</b> | 78.69        | 80.18        |
| CWE-120 | <b>95.17</b>                       | <b>78.15</b> | 77.9         | 78.02        | 94.7                  | 71.36        | <b>87.38</b> | <b>78.56</b> |
| CWE-125 | 85.82                              | 60.58        | 65.22        | 62.81        | <b>92.59</b>          | 64.11        | <b>82.17</b> | <b>72.03</b> |
| CWE-190 | 95.49                              | 94.91        | 97.3         | 96.09        | <b>99.32</b>          | <b>98.72</b> | <b>97.26</b> | <b>97.98</b> |
| CWE-399 | 84.99                              | 79.72        | <b>82.56</b> | <b>81.15</b> | 84.42                 | 76.91        | 81.99        | 79.37        |
| CWE-400 | <b>90.93</b>                       | 63.39        | 67.48        | 65.37        | 89.97                 | 69.24        | <b>72.57</b> | <b>70.87</b> |
| CWE-787 | 92.49                              | 79.76        | 77.34        | 78.53        | <b>96.96</b>          | <b>84.66</b> | 82.32        | <b>83.47</b> |
| Average | 92.49                              | 77.9         | 79.18        | 78.53        | <b>94.32</b>          | <b>78.45</b> | <b>82.31</b> | <b>80.12</b> |

Bold values indicate the maximum values of four metrics for a vulnerability type under different graph settings

This table compares the performance contributions of different self-defined edges in VulDegre

vulnerabilities such as CWE-74 and CWE-77, and sometimes the performance may even decrease. The F1 scores for CWE-74 and CWE-77 decrease by 1.94% and 2.13%, respectively. We believe this is because these vulnerabilities are not dependent on variable and parameter types. Adding variable-type information may increase the number of nodes the graph neural network needs to process, leading to inaccurate or irrelevant feature representations. Therefore, adding variable-type information nodes may hinder the model's ability to identify and learn the patterns associated with these vulnerabilities.

Additionally, adding ParameterDependency edges significantly improves the detection performance of vulnerabilities dependent on parameter and variable relationships, such as Command Injection (CWE-77) and Operating System Command

Injection (CWE-78) vulnerabilities—two of the most common types of injection vulnerabilities. For instance, the F1 score for CWE-77 increases by 6.28%. Moreover, ParameterDependency offers broad performance improvements in detecting vulnerabilities related to variable and buffer types. For example, the detection of the CWE-190 vulnerability saw a 7.44% increase in the F1 score. However, for vulnerability types arising from resource management errors, such as CWE-399 and CWE-400, the performance improvement is minimal, and in some cases, the metrics even show negative growth. Figure 9 illustrates a classic example of a CWE-399 (resource management error) vulnerability, with the CVE number of CVE-2011–3192. This code demonstrates a system resource management issue when Apache handles HTTP requests involving multiple overlapping byte ranges. Specifically, on line 7, the server loads the appropriate portion of each file range into memory and sends a response without verifying the total memory consumed. Even though memory is freed on line 10, the damage caused by resource exhaustion may have already occurred. Our method struggles to capture this type of vulnerability effectively. We believe this is because it involves dynamic resource management, and our graph structure fails to capture how different inputs can lead to vastly different execution paths and resource usage patterns.

When two edges are added to the graph structure, performance improvements were observed for all vulnerabilities except those related to resource management. This demonstrates that adding both edges simultaneously can effectively compensate for their individual limitations and achieve overall performance gains, and it also illustrates the versatility of VulDecgre in detecting multiple types of vulnerabilities.

In conclusion, adding the TypeInformation and ParameterDependency relationships to the code graph structure effectively represents vulnerability characteristics related to parameter dependencies and type information, thereby improving the model's performance. However, adding these two types of information may have side effects for detecting certain types of vulnerabilities.

### 5.3 RQ3: Effectiveness of NS-GGNN

To address this research question, we compared the detection performance of the proposed NS-GGNN module against three high-performing graph-based neural network methods using the ten types of vulnerabilities mentioned above. The first method uses the GGNN module from Devign et al. (2019), and the second method is from VulCNN (2022), which is an efficient CNN module. The third method uses the GGNAT module from EnGS2F (2024), which incorporates a gated graph recurrent layer and a graph attention layer. We performed experiments on the three modules under the same conditions. Specifically, we represented the code as the NAST structure graph and tested all modules. By comparing the impact of different modules on the detection of ten types of vulnerabilities in our dataset, we analyzed the effect of the NS-GGNN module on VulDecgre's performance.

As shown in Table 5, the NS-GGNN module improves VulDecgre's performance across all types of vulnerabilities in the dataset. For instance, regarding F1 scores, the NS-GGNN module achieves an average improvement of 8.17% over the VulCNN

module across ten vulnerabilities. Compared to the Devign method, which uses the original GGNN, the NS-GGNN module achieves an average improvement of 25.55% in F1 scores. Compared with the GGNAT module that uses graph attention based on the GGNN module, the NS-GGNN module achieves 3.89% performance improvement. Additionally, the NS-GGNN module significantly outperforms the other three modules in detecting vulnerabilities with clear patterns, such as overflow and injection vulnerabilities. For instance, when detecting CWE-78 (OS Command Injection) vulnerabilities, the NS-GGNN module achieves performance improvements of 24.27%, 18.48%, and 6.38% compared to the other three modules in terms of F1 score.

Furthermore, the NS-GGNN module achieves the best performance in at least two metrics for each vulnerability type, demonstrating its effectiveness across various vulnerabilities. We also found that the NS-GGNN module improves most metrics compared to the GGNAT module. Specifically, regarding the accuracy and F1 score, NS-GGNN improved by 4.3% and 3.89% compared to the GGNAT module, respectively. Although GGNAT also uses a gate graph attention layer similar to NS-GGNN to capture the code graph structure effectively. It cannot guarantee sufficient capture of remote execution sequence information, resulting in inadequate module performance. Figure 10 shows the front part of the real-world command injection vulnerability function, omitting about 50 lines of the rest. It can be triggered if the ENV\_VARIABLE environment variable contains malicious input, which is appended to the data\_buf and subsequently executed by the SYSTEM() function. Among all the methods, only VulDecgre successfully detects this vulnerability. The main reasons are as follows: First, VulDecgre's code graph incorporates parameter dependencies, providing a more comprehensive representation of the variables involved in SYSTEM functions that trigger vulnerabilities. Second, traditional GNN modules lose the dependency relationships related to vulnerabilities on the leftmost side of the code AST when learning long-range code representations, while NS-GGNN can address this issue.

In conclusion, these results indicate that the NS-GGNN module outperforms other modules in detecting various vulnerabilities.

## 5.4 RQ4: Influence of hyperparameters on VulDecgre

Tuning parameters and hyperparameters plays a critical role in the performance of VulDecgre for vulnerability detection. We investigated the influence of hyperparameters in the VulDecgre model, including the number of layers in the gated graph neural network and the number of attention heads in the NS-GGNN module. We utilized Neural Network Intelligence (NNI) to fine-tune these hyperparameters.

### 5.4.1 Numbers of GGNN layers

Table 6 illustrates the effect of varying the number of graph neural network layers on VulDecgre's performance in a mixed dataset of 10 vulnerabilities. The NS-GGNN achieves its best performance, with 95.42% accuracy when the number of layers is set to 5. Additionally, the model's accuracy improves as the number of layers

**Table 5** Comparison of NS-GGNN and state-of-the-art module performance

| CWE-ID  | GGNN     |           |              |       | VulCNN   |           |              |       |
|---------|----------|-----------|--------------|-------|----------|-----------|--------------|-------|
|         | Accuracy | Precision | Recall       | F1    | Accuracy | Precision | Recall       | F1    |
| CWE-74  | 71.88    | 42.63     | 71.05        | 53.29 | 90.36    | 63.11     | 86.4         | 72.95 |
| CWE-77  | 47.66    | 22.61     | 83.74        | 35.61 | 80.13    | 52.12     | 64.38        | 57.60 |
| CWE-78  | 71.91    | 48.8      | 57.01        | 52.59 | 82.17    | 69.56     | 50.3         | 58.38 |
| CWE-119 | 78.47    | 44.05     | 65.49        | 52.67 | 90.59    | 63.43     | 86.74        | 73.28 |
| CWE-120 | 75.31    | 43.85     | 53.27        | 48.1  | 81.36    | 69.7      | 74.52        | 72.03 |
| CWE-125 | 88.99    | 60.12     | 65.16        | 62.54 | 84.47    | 58.45     | 51.83        | 54.94 |
| CWE-190 | 39.06    | 37.57     | <b>99.94</b> | 54.61 | 91.64    | 93.62     | 98.33        | 95.91 |
| CWE-399 | 78.82    | 60.52     | 89.62        | 72.25 | 85.36    | 77.4      | <b>95.39</b> | 85.46 |
| CWE-400 | 52.54    | 17.58     | 53.05        | 26.41 | 84.67    | 73.67     | 70.22        | 71.9  |
| CWE-787 | 88.21    | 63.6      | 76.6         | 69.50 | 79.58    | 64.26     | 79.08        | 70.9  |
| Average | 69.29    | 44.13     | 71.49        | 54.57 | 85.03    | 68.53     | 75.72        | 71.95 |

| CWE-ID  | GGNAT    |              |              |              | NS-GGNN      |              |              |              |
|---------|----------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|         | Accuracy | Precision    | Recall       | F1           | Accuracy     | Precision    | Recall       | F1           |
| CWE-74  | 94.88    | 80.63        | 85.35        | 82.92        | <b>97.37</b> | <b>83.89</b> | <b>86.7</b>  | <b>85.27</b> |
| CWE-77  | 90.21    | 72.19        | <b>86.47</b> | 78.69        | <b>93.35</b> | <b>72.25</b> | 81.43        | <b>76.57</b> |
| CWE-78  | 93.37    | <b>84.23</b> | 60.59        | 70.48        | <b>97.09</b> | 81.63        | <b>72.62</b> | <b>76.86</b> |
| CWE-119 | 93.5     | 77.29        | <b>90.64</b> | 83.43        | <b>97.44</b> | <b>81.73</b> | 78.69        | <b>80.18</b> |
| CWE-120 | 88.74    | 65.3         | 70.27        | 67.7         | <b>94.7</b>  | <b>71.36</b> | <b>87.38</b> | <b>78.56</b> |
| CWE-125 | 90.78    | <b>70.27</b> | <b>68.85</b> | 69.55        | <b>92.59</b> | 64.11        | 82.17        | <b>72.03</b> |
| CWE-190 | 93.26    | 90.12        | 92.1         | 91.09        | <b>99.32</b> | <b>98.72</b> | 97.26        | <b>97.98</b> |
| CWE-399 | 80.66    | 74.3         | 80.25        | <b>77.16</b> | <b>84.42</b> | <b>76.91</b> | 81.99        | 79.37        |
| CWE-400 | 80.24    | 60.5         | 64.67        | 70.39        | <b>89.97</b> | <b>69.24</b> | <b>72.57</b> | 70.87        |
| CWE-787 | 94.51    | 80.74        | 69.98        | 74.98        | <b>96.96</b> | <b>84.66</b> | <b>82.32</b> | <b>83.47</b> |
| Average | 90.02    | 75.56        | 76.92        | 76.23        | <b>94.32</b> | <b>78.45</b> | <b>82.31</b> | <b>80.12</b> |

Bold values indicate the maximum values of the four measures of a vulnerability type under different graph modules

This table compares the performance of different graph-based vulnerability detection modules

```

1 void process_byte_range_request(request_t *req) {
2     ...
3     list_t *ranges = parse_byte_ranges(req->headers["Range"]);
4     file_t *file = open_file(req->url);
5     ...
6     foreach(range in ranges) {
7         byte_t *data = load_file_segment(file, range.start, range.end);
8         send_response(req, data, range.end - range.start + 1);
9         // Freeing the data after sending, but the damage may already be done.
10        free(data);
11    }
12    close_file(file);
13    ...
14 }

```

**Fig. 9** An example of the CWE-399 vulnerability, which our method finds difficult to detect

```

1 void execute_command_from_environment()
2 {
3     wchar_t * data;
4     wchar_t data_buf[100] = FULL_COMMAND;
5     data = data_buf;
6     size_t dataLen = wcslen(data);
7     ...
8     wchar_t * environment = GETENV(ENV_VARIABLE);
9     if (environment != NULL){
10         wcsncat(data+dataLen, environment, 100-dataLen-1);
11     }
12     if (SYSTEM(data) != 0){
13         printLine("command execution failed!");
14         exit(1);
15     }
16     ...
17 }

```

**Fig. 10** A real-world command injection vulnerability (CWE-78) function, which is successfully detected only by our method

**Table 6** Comparison of VulDegre hyperparameters

| Layer number | Accuracy     | Head number | Accuracy     |
|--------------|--------------|-------------|--------------|
| 2            | 92.19        | 2           | 90.66        |
| 4            | 93.87        | 4           | 95.14        |
| 5            | <b>95.42</b> | 6           | 94.7         |
| 6            | 94.55        | 8           | <b>95.42</b> |
| 8            | 89.29        | 10          | 95.26        |

Bold values indicate the maximum accuracy for different hyperparameter settings

increases up to 5, indicating that a deeper network structure can learn more complex relationships within the code graph structure. However, when the number of layers exceeds 5, the model's accuracy decreases as the number of layers increases. We believe that an excessive number of layers significantly increases the complexity of graph data, introducing substantial noise that interferes with VulDegre's effectiveness.

#### 5.4.2 Numbers of attention heads

Table 6 also shows VulDegre's performance with varying numbers of attention heads. Overall, increasing the number of attention heads improves VulDegre's accuracy, as more attention heads can capture more comprehensive code tokens. When the number of heads is increased to 8, the model achieves its best accuracy of 95.42%. However, when the number of heads exceeds 8, the model's performance no longer improves because the new head information overlaps with the existing head information, resulting in diminishing returns.

In conclusion, VulDecgre achieves the best vulnerability detection results when the number of GGNN layers is set to 5, and the number of attention heads is set to 8.

## 6 Related work

### 6.1 Graph-based code representation methods

Unlike natural languages, programming languages are more structured and hierarchical, encompassing various dimensions such as abstract syntax trees, data flow, and control flow. Graph-based code representation can capture program source code's complex structural and semantic information. One commonly used method represents the code structure from different aspects, including integrating code graphs and performing multi-angle feature extraction. Zhang et al. (2023) constructed a slice relation graph by adding node relationships relevant to key function contexts to the program dependence graph, enabling the extraction of features from nodes associated with vulnerable functions for program repair. Wang et al. (2023) proposed the DeepVD method, which separately extracts features from exception flow graphs, post-dominator trees, call relation graphs, long paths in the AST, and natural code sequences. Zhang et al. (2023) extracted paths composed of nodes associated with common defect functions on the CFG graph to achieve path-level vulnerability detection. Many studies have used similar methods (Liu et al. 2024; Ni et al. 2023). These graphs can effectively represent general code structure information, but they are still insufficient to represent some vulnerability codes with distinct structural characteristics.

Another common approach to graph representation involves customizing specific edge relationships for particular tasks based on the original code graph structure. Wang et al. (2020) introduced the FUNDED method, which constructs a fused code graph structure by adding six types of edge relationships to the graph for specific vulnerability types, such as Jump and ComputedFrom edges. Niu et al. (2024) proposed FAIR, which adds a custom call relationship edge between control flow graphs of different functions to capture call dependencies and represent code across multiple functions. Zhu et al. (2023) proposed the Tare method, which constructs a graph structure T-Graph with multiple custom edge relationships to capture code information. Similar techniques have been used in other studies (Yang et al. 2022; Liu et al. 2023). Inspired by these graph structures, we constructed a graph structure that integrates multi-class code graph structures and self-defined edge relationships to effectively represent the characteristics of vulnerability code and achieve high-precision vulnerability detection.

### 6.2 Deep learning-based vulnerability detection

Deep learning-based research is a prominent topic in vulnerability detection. Early deep learning-based vulnerability detection methods treat source code fragments as

flat, linear sequences and vectorize them using word embedding techniques, such as Word2Vec, to learn code features through temporal networks. Saccente et al. (2019) introduced Project Achilles, a prototype tool that reads function code as token sequences and uses LSTM to learn the code sequences. Several other studies have also used LSTM as an encoder for code (Wartschinski et al. 2022; Zhou et al. 2023).

In recent years, many methods have explored GNNs to delve deeper into the structural information in code and have achieved significant progress in vulnerability detection. DeepWukong (2021) used GCN and GAT to process the XFG graph structure generated from source code slicing for vulnerability prediction. Devign (2019) and Reveal (2021) applied GGNN to analyze code graphs. Wang et al. (2020) proposed the FUNDED method, which introduced six new types of edges based on the AST to enhance graph representation learning. Wen et al. (2024) introduced the LIVABLE method, which employs a differentiated propagation-based GNN to analyze code graphs. Wen et al. (2023) also proposed the AMPLE model, which combines code node simplification and graph node enhancement methods for accurate vulnerability detection. Although these methods can achieve good vulnerability detection performance, they inevitably lose key information necessary for the model to understand code behavior, such as the code execution order, while learning the vulnerability characteristics.

Additionally, several studies have employed Transformer-based approaches to detect vulnerabilities. Fu et al. (2023) proposed the OPTIMATCH method, which clusters a vulnerability codebook to represent the patterns of each vulnerability and employs a Transformer-based model to detect vulnerabilities at both the statement and function levels. However, these methods lack contextual program structure information in the constructed graph features, limiting their ability to capture contextual features during graph representation learning. Therefore, we leveraged the advantages of graph representation learning and addressed the shortcomings of graph networks to construct a deep learning-based vulnerability detection method.

## 7 Threats and limitations

The experiments were carefully designed, and the data were rigorously verified. However, several challenges may still affect the validity of the obtained results.

The first threat to validity is the low and imbalanced proportion of vulnerability samples for each type in conventional datasets. To address this issue, we constructed a dataset with a large number of samples, diverse types, a high vulnerability ratio, and a balanced distribution of positive and negative samples. Compared to the three commonly used datasets, FFMpeg+Qemu, Reveal et al., and Fan et al., VulDecgre has significantly improved detection performance on our dataset.

The second threat to validity is that our study collected only C and C++ language data, focusing exclusively on vulnerability detection in C and C++ and not considering vulnerabilities in other languages, such as Python and Java. This greatly limits the cross-language capabilities of the model.

The third threat to validity is that VulDecgre performs poorly and requires excessive training time when detecting vulnerabilities in graph nodes with a high number

of tokens. This is because the non-terminal nodes of the AST generate computational redundancy during the graph representation process, and VulDecgre requires additional time to learn the relationships between code structures, thereby diminishing the model's performance. In our experiments, we limited the maximum number of tokens to 500 to mitigate this issue. In the future, we will employ program reduction techniques to simplify the nodes and relationships in the code graph, thereby reducing the training time.

Moreover, in the design of the NS-GGNN module, we only process the sequences connected through the NextToken edges to mitigate the loss of long-range dependency information, but the loss of other types of information remains and cannot be overlooked. However, due to the diversity and complexity of code structures, certain types of edges may not be present in the code graph, making it difficult to effectively capture them.

## 8 Future work

In the future, we will explore extracting and processing additional types of information from the code graph to mitigate the loss of long-range dependencies better. To enhance VulDecgre's ability to detect challenging vulnerabilities (e.g., resource management vulnerabilities), we plan to explore combining it with dynamic execution strategies and static analysis frameworks to improve the efficiency of detecting resource management vulnerabilities during dynamic execution. Specifically, we plan to incorporate symbolic execution into the code representation process to capture execution path characteristics, addressing the absence of dynamic execution capabilities in VulDecgre. We will also integrate VulDecgre into code analysis tools Cppcheck<sup>7</sup> by utilizing its interface implementations to analyze specific functions or code blocks with VulDecgre. This integration aims to enhance Cppcheck's capability to detect a wider range of vulnerabilities. We also plan to extend VulDecgre to support Java, Python, and JavaScript and collect function-level vulnerability datasets in other languages to verify VulDecgre's effectiveness on these datasets. Additionally, our work has demonstrated the effectiveness of VulDecgre in real-world C/C++ vulnerability detection, where it can identify multiple types of vulnerabilities with high stability. We aim to collaborate with our industry partners to deploy VulDecgre in a proprietary security system, testing the security of user-supplied application source code and providing vulnerability feedback to evaluate its effectiveness in real-world applications. We plan to deploy the VulDecgre model on the backend of an existing online software vulnerability detection platform via its interface. This setup will enable VulDecgre to perform vulnerability analysis on code uploaded through the web interface. By integrating VulDecgre with the prevailing vulnerability detection models, we will leverage its unique strengths to provide more comprehensive and enhanced performance in vulnerability detection. In addition, we will also explore the use of VulDecgre in mobile applications or web services in the future.

---

<sup>7</sup> <https://cppcheck.sourceforge.io>.



## 9 Conclusion

This paper proposes a new method (i.e., VulDecgre) for detecting vulnerabilities based on enhanced graph representation learning. VulDecgre incorporates multiple relational edges into traditional code graphs, enabling the newly constructed graph structure to represent the contextual information of functions more effectively. The graph node embeddings and execution sequence node embeddings effectively capture the code's global and long-distance dependency features. Additionally, GGNN is combined with a multi-head attention mechanism to improve the model's performance in vulnerability detection tasks. VulDecgre is applied to detect vulnerabilities at the function level across three public datasets and one self-collected dataset. Experimental results demonstrate that VulDecgre outperforms current baseline methods. Compared to state-of-the-art deep learning-based approaches, VulDecgre improves vulnerability detection performance across all datasets by 0.8–33.55% in terms of F1 score. On our dataset, VulDecgre has an average accuracy and F1 score of 93.32% and 83.04% across 18 categories of vulnerabilities, achieving the best vulnerability detection results. In the future, we will explore expanding VulDecgre to support additional languages and integrate it into broader software security applications.

**Acknowledgements** This work is sponsored by the National Natural Science Foundation of China (Nos. 61973180, 62172249, 62202253), the Natural Science Foundation of Shandong Province (Nos. ZR2021MF092, ZR2021QF074, ZR2022MF326, ZR2024QF199).

**Data Availability** [https://drive.google.com/file/d/1P0NsDzpL75g5-EKJ59qCYK384UE2ohZ5/view?usp=drive\\_link](https://drive.google.com/file/d/1P0NsDzpL75g5-EKJ59qCYK384UE2ohZ5/view?usp=drive_link).

**Code availability** <https://github.com/SoftEngineerTeam/VulDegre>.

## References

- Ahmad, W.U., Chakraborty, S., Ray, B., Chang, K.-W.: Unified pre-training for program understanding and generation. arXiv preprint [arXiv:2103.06333](https://arxiv.org/abs/2103.06333) (2021)
- Alon, U., Yahav, E.: On the bottleneck of graph neural networks and its practical implications. arXiv preprint [arXiv:2006.05205](https://arxiv.org/abs/2006.05205) (2020)
- Cadar, C., Dunbar, D., Engler, D.R., et al.: Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In: OSDI, vol. 8, pp. 209–224 (2008)
- Chung, J., Gulcehre, C., Cho, K., Bengio, Y.: Empirical evaluation of gated recurrent neural networks on sequence modeling. arXiv preprint [arXiv:1412.3555](https://arxiv.org/abs/1412.3555) (2014)
- Church, K.W.: Word2vec. Nat. Lang. Eng. **23**(1), 155–162 (2017)
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B.: Deep learning based vulnerability detection: are we there yet? IEEE Trans. Softw. Eng. **48**(9), 3280–3296 (2021)
- Cao, S., Sun, X., Wu, X., Lo, D., Bo, L., Li, B., Liu, W.: Coca: improving and explaining graph neural network-based vulnerability detection systems. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13 (2024)
- Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y.: Deepwukong: statically detecting software vulnerabilities using deep graph neural network. ACM Trans. Softw. Eng. Method. **30**(3), 1–33 (2021)

- Cheng, X., Zhang, G., Wang, H., Sui, Y.: Path-sensitive code embedding via contrastive learning for software vulnerability detection. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 519–531 (2022)
- Dam, H.K., Pham, T., Ng, S.W., Tran, T., Grundy, J., Ghose, A., Kim, T., Kim, C.-J.: Lessons learned from using a deep tree-based model for software defect prediction in practice. In: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), pp. 46–57. IEEE (2019)
- Feng, Q., Feng, C., Hong, W.: Graph neural network-based vulnerability predication. In: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 800–801. IEEE (2020)
- Fu, M., Le, T., Nguyen, V., Tantithamthavorn, C., Phung, D.: Learning to quantize vulnerability patterns and match to locate statement-level vulnerabilities. arXiv preprint [arXiv:2306.06109](https://arxiv.org/abs/2306.06109) (2023)
- Fan, J., Li, Y., Wang, S., Nguyen, T.N.: Ac/c++ code vulnerability dataset with code changes and cve summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories, pp. 508–512 (2020)
- Fan, Y., Wan, C., Fu, C., Han, L., Xu, H.: Vdotr: vulnerability detection based on tensor representation of comprehensive code graphs. *Comput. Secur.* **130**, 103247 (2023)
- Gao, Q., Ma, S., Shao, S., Sui, Y., Zhao, G., Ma, L., Ma, X., Duan, F., Deng, X., Zhang, S., et al.: Cobot: static c/c++ bug detection in the presence of incomplete code. In: Proceedings of the 26th Conference on Program Comprehension, pp. 385–388 (2018)
- Ghaffarian, S.M., Shahriari, H.R.: Neural software vulnerability analysis using rich intermediate graph representations of programs. *Inf. Sci.* **553**, 189–207 (2021)
- Li, Y., Tarlow, D., Brockschmidt, M., Zemel, R.: Gated graph sequence neural networks. arXiv preprint [arXiv:1511.05493](https://arxiv.org/abs/1511.05493) (2015)
- Li, Q., Han, Z., Wu, X.-M.: Deeper insights into graph convolutional networks for semi-supervised learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018a)
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y.: Vuldeepecker: a deep learning-based system for vulnerability detection. arXiv preprint [arXiv:1801.01681](https://arxiv.org/abs/1801.01681) (2018b)
- Li, Y., Wang, S., Nguyen, T.N.: Vulnerability detection with fine-grained interpretations. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 292–303 (2021a)
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z.: Sysevr: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secure Comput.* **19**(4), 2244–2258 (2021b)
- Liu, S., Xie, X., Siow, J., Ma, L., Meng, G., Liu, Y.: Graphsearchnet: enhancing gnn's via capturing global dependencies for semantic code search. *IEEE Trans. Softw. Eng.* **49**(4), 2839–2855 (2023)
- Liu, H., Jiang, S., Qi, X., Qu, Y., Li, H., Li, T., Guo, C., Guo, S.: Detect software vulnerabilities with weight biases via graph neural networks. *Expert Syst. Appl.* **238**, 121764 (2024)
- Ni, C., Yin, X., Yang, K., Zhao, D., Xing, Z., Xia, X.: Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 1611–1622 (2023)
- Niu, C., Li, C., Ng, V., Lo, D., Luo, B.: Fair: flow type-aware pre-training of compiler intermediate representations. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–12 (2024)
- Oono, K., Suzuki, T.: Graph neural networks exponentially lose expressive power for node classification. arXiv preprint [arXiv:1905.10947](https://arxiv.org/abs/1905.10947) (2019)
- Peng, Y., Wang, C., Wang, W., Gao, C., Lyu, M.R.: Generative type inference for python. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 988–999. IEEE (2023)
- Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M.: Automated vulnerability detection in source code using deep representation learning. In: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), pp. 757–762. IEEE (2018)
- Saccante, N., Dehlinger, J., Deng, L., Chakraborty, S., Xiong, Y.: Project achilles: a prototype tool for static method-level vulnerability detection of java source code using a recurrent neural network. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW), pp. 114–121. IEEE (2019)
- Song, C., Wang, X., Zhang, W.: Analysis and optimization of angr in dynamic software test application. *Comput. Eng. Sci.* **40**(S1), 167–172 (2018)

- Tang, X., Tian, H., Chen, Z., Pian, W., Ezzini, S., Kabore, A.K., Habib, A., Klein, J., Bissyande, T.F.: Learning to represent patches. *arXiv preprint arXiv:2308.16586* (2023)
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Adv. Neural. Inf. Process. Syst.* **30**(1), 261–272 (2017)
- Wen, X.-C., Chen, Y., Gao, C., Zhang, H., Zhang, J.M., Liao, Q.: Vulnerability detection with graph simplification and enhanced graph representation learning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2275–2286. IEEE (2023)
- Wen, X.-C., Gao, C., Luo, F., Wang, H., Li, G., Liao, Q.: Livable: exploring long-tailed classification of software vulnerability types. *IEEE Trans. Softw. Eng.* (2024)
- Wartschinski, L., Noller, Y., Vogel, T., Kehrer, T., Grunske, L.: Vudenc: vulnerability detection with deep learning on a natural codebase for python. *Inf. Softw. Technol.* **144**, 106809 (2022)
- Wang, H., Ye, G., Tang, Z., Tan, S.H., Huang, S., Fang, D., Feng, Y., Bian, L., Wang, Z.: Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans. Inf. Forensics Secur.* **16**, 1943–1958 (2020)
- Wang, W., Nguyen, T.N., Wang, S., Li, Y., Zhang, J., Yadavally, A.: Deepvd: toward class-separation features for neural network vulnerability detection. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2249–2261. IEEE (2023)
- Wang, H., Tang, Z., Tan, S.H., Wang, J., Liu, Y., Fang, H., Xia, C., Wang, Z.: Combining structured static code information and dynamic symbolic traces for software vulnerability prediction. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13 (2024)
- Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H.: Vulcnn: an image-inspired scalable vulnerability detection system. In: Proceedings of the 44th International Conference on Software Engineering, pp. 2365–2376 (2022)
- Xiao, P., Xiao, Q., Zhang, X., Wu, Y., Yang, F.: Vulnerability detection based on enhanced graph representation learning. *IEEE Trans. Inf. Forensics Secur.* (2024)
- Yuan, B., Lu, Y., Fang, Y., Wu, Y., Zou, D., Li, Z., Li, Z., Jin, H.: Enhancing deep learning-based vulnerability detection by building behavior graph model. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 2262–2274. IEEE (2023)
- Yang, D., Mao, X., Chen, L., Xu, X., Lei, Y., Lo, D., He, J.: Transplantfix: graph differencing-based code transplantation for automated program repair. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–13 (2022)
- Zhang, Y., Li, G., Jin, Z., Xing, Y.: Neural program repair with program dependence analysis and effective filter mechanism. *arXiv preprint arXiv:2305.09315* (2023)
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Adv. Neural Inf. Process. Syst.* **32**, 10197–10207 (2019)
- Zhou, S., Li, J., Wang, H., Shang, S., Han, P.: Grlstm: trajectory similarity computation with graph-based residual lstm. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, pp. 4972–4980 (2023)
- Zhu, Q., Sun, Z., Zhang, W., Xiong, Y., Zhang, L.: Tare: type-aware neural program repair. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 1443–1455. IEEE (2023)
- Zhang, J., Wang, X., Zhang, H., Sun, H., Liu, X., Hu, C., Liu, Y.: Detecting condition-related bugs with control flow graph neural network. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1370–1382 (2023)

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

## Authors and Affiliations

Xuehai Jia<sup>1,2</sup> · Junwei Du<sup>2</sup> · Minying Fang<sup>2</sup> · Hao Liu<sup>2</sup> · Yuying Li<sup>2</sup> · Feng Jiang<sup>1,2</sup>

✉ Yuying Li  
liyy@qust.edu.cn

✉ Feng Jiang  
jiangfeng@qust.edu.cn

Xuehai Jia  
timd686@gmail.com

Junwei Du  
djwqd@163.com

Minying Fang  
qustfmy@163.com

Hao Liu  
upcliu010118@gmail.com

<sup>1</sup> College of Information Science and Technology, Qingdao University of Science and Technology, Qingdao 266061, China

<sup>2</sup> College of Data Science, Qingdao University of Science and Technology, Qingdao 266061, China